

## I) INTRODUCTION TO ALGORITHMS

### Algorithm definition :-

- An algorithm is a finite set of instructions or logic written in order, to accomplish a certain predefined task.
- An algorithm is a collection of clear instructions for solving a specific issue in computer programming.
- A step by step problem solving procedure.
- A sequence of instructions that tell how to solve a particular problem.

### Characteristics / Properties of an Algorithm

5M  
Imp

Algorithm is best way to represent a solution of particular problem in a simple and efficient way

1. Input :- What the algorithm take as input.

- An algorithm should be have 0 or more well defined inputs

2. Output :- what the algorithm produce as output  
Algorithm should have at least one output. or  
algorithm should have one or more well defined outputs.

3. Definiteness :- Each instruction is clear and precised and easy to interpret that means specifying the sequence of operations for turning input into output.

4. Finiteness :- algorithm should have limited number of instructions that instructions should be countable.

5. Effectiveness :- Every instruction must be very basic, so that it can be carried out by a

person using pencil & paper.   
 → computer science & software development   
 → navigation app.   
 → social media   
 → virtual assistant   
 → health care   
 → fraud detection   
 → engineering   
 → data science   
 → AI

6. Feasibility: should be feasible with available resource.

21/9/24

Advantages of Algorithms

- It is easy to understand.
- An algorithm is a step-wise representation of a solution to a given problem.
- It is easier for the programmer to convert it into an actual program.

Disadvantages of Algorithms

- Writing an algorithm takes a long time so it is time consuming.   
 • complexity - for larger program   
 • not suitable for all problems   
 • errors

Example:

Algorithm to add 3 numbers and print their sum

- START
- Declare 3 integer variables num1, num2, and num3.
- Take the three numbers to be added as inputs in variables num1, num2 and num3 respectively.
- Declare an integer variable sum to store the resultant sum of the 3 numbers.
- Add the 3 numbers and store the result in the variable sum.
- Print the value of variable sum.
- End

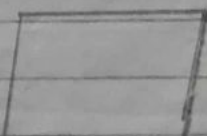
## Algorithm Design:

- There are two common methods to express algorithm designs, they are pseudocode and flowcharts.
- Our pseudocode page explains about expressing algorithms as pseudocode.
- Our Flowcharts page explains about expressing algorithms as flowcharts.
- Both methods are perfectly acceptable and have their individual merits.

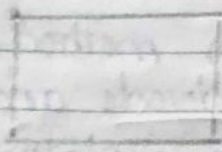
## FLOW CHART

- Flow chart is a graphical or diagrammatical representation of algorithm.
- A very effective tool to show the logic flow of a program is the flowchart.
- A flowchart is a pictorial & schematic representation of an algorithm or a stepwise process, showing the steps as boxes of various kinds and their order by connecting these with arrows.
- Flowchart are used in designing or documenting a process or program.
- Flowcharts are usually drawn using some symbols.

## Flowchart Symbols & Descriptions

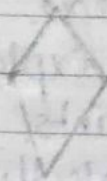
| Name                   | Symbol                                                                              | Description.                                                     |
|------------------------|-------------------------------------------------------------------------------------|------------------------------------------------------------------|
| ① oval.                |  | Indicates starting or ending of flowcharts, (START / STOP (END)) |
| ② connector            |  | use to connect break in a chart.                                 |
| ③ Parallelogram<br>I/O |  | Used for Input & output operations                               |

④ Rectangle



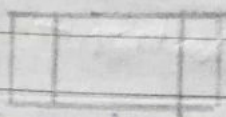
Indicate any interval operation or mathematical operation.

⑤ Diamond



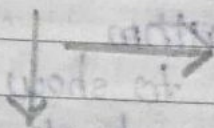
Use for asking that can have either true or false (Yes/no)

⑥ Module



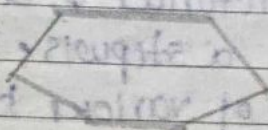
Sub soln to solve a part of entire problem.

⑦ control flow / arrows



Show the flow of direction.

⑧ For loop



preparation.

\* Program to check a person is eligible for vote or not

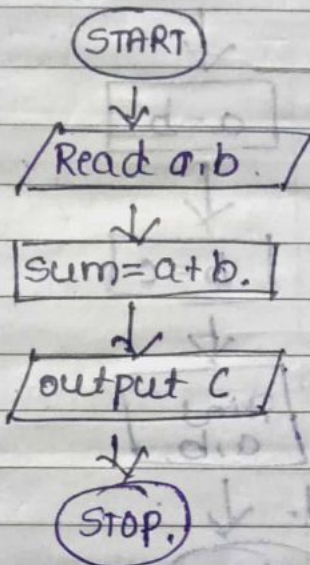
```
#include <stdio.h>
#include <conio.h>
void main( )
{
    int age;
    printf("enter age");
    scanf("%d", &age);
    if (age >= 18)
    {
        printf("eligible to vote");
    }
    else
    {
        printf("not eligible to vote.");
    }
    getch();
}
```

no need of flower bracket

Example:

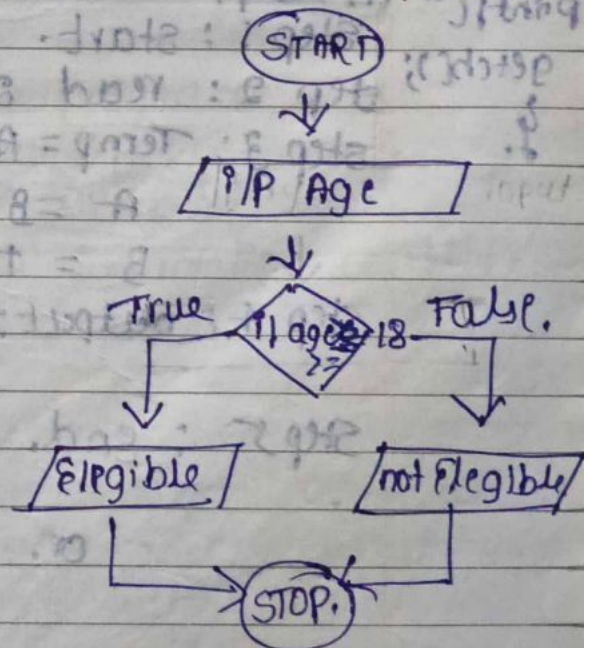
- ① Write an algorithm for Sum of two numbers draw the flow chart for same,

1. Start.
2. Read a, b (input)
3. Sum  $a+b$  (calculation)
4. Output c
5. Stop.



- ② Write an algorithm and flowchart whether a person is eligible for vote or not.

- step 1: Start
- step 2: input age
- step 3: if (age  $\geq 18$ )  
Print "Elegible"  
else "Not Elegible"
- step 4: ~~stop~~  
"Not elegible"
- step 4: stop



③ Write an algorithm & flowchart to swap two numbers.

Algorithm  
 Step 1: Start.  
 Step 2: Read a, b, c (input).  
 Step 3:  $c = a$ .  
 Step 4:  $b = b$ .  
 Step 5:  $b = c$ .  
 Step 6: Print a and b.  
 Step 7: Stop.

```

#include <stdio.h>
int main()
{
    int a, b, temp = 0;
    printf("Enter the values for a & b");
    scanf("%d %d", &a, &b);
    printf("Before swapping a = %d, b = %d", a, b);
    temp = a;
    a = b;
    b = temp;
    printf("After swapped values are a = %d, b = %d", a, b);
}
  
```

Flowchart:  
 Start → Read a, b →  $c = a$  →  $b = b$  →  $b = c$  → Print a and b → Stop.

1: Start

2: Input a, b

3: Declare 3rd variable c

4: Calculate  $c = a$   
 $a = b$

$c = a$   
 $a = b$   
 $b = c$

5: Print a, b

Stop

23/9/24  
 Pseudocode - only used for designing program language.

③ Pseudo code is a method of expressing an algorithm design.

④ It is compact and informal high level description program.

⑤ Used for only planning of program.

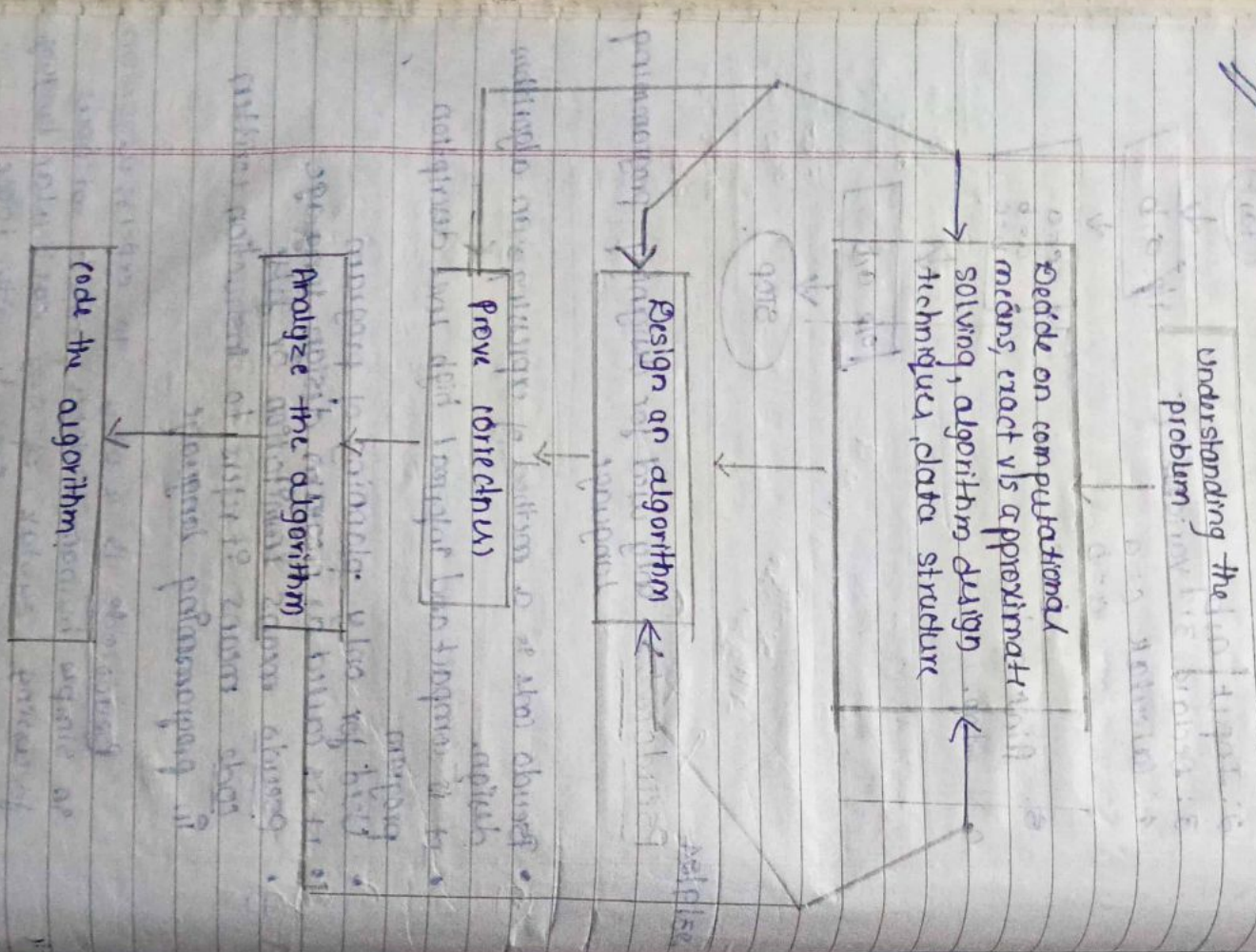
⑥ It is called as program design language.

⑦ Pseudo means imitation or false.  
 code means refers to instruction written in programming language.

Pseudocode is a way to express algorithm in simple human readable format without focussing syntax of any particular language using plain English with logic structure.

once pseudocode is clear it can be translated into any language including C.

## Problem Solving aspects



## Problem Solving aspects/implementation algorithm/ basic steps for designing algorithm/ concept of algorithm

### 1. Understand the Problem

- Read the problem
- Ask questions
- Find and decide the range of inputs

### 2. Make Decision

- select problem solving method exact & approximate
- Data structure
- algorithmic strategies

### 3. Design an algorithm

- Natural language (English)
- Pseudocode
- Flowchart

### 4. Prove correctness

- verification
- validation  $\rightarrow$   $\frac{\text{Writing}}{\text{effectiveness of algorithm}}$

### 5. Analyze the algorithm

- Time complexity
- Space complexity
- Simplicity  $\rightarrow$   $\frac{\text{reducing complexity of algorithm}}{\text{design in easy way}}$

### 6. Code the algorithm

- Suitable programming language
- Writing optimized code  $\rightarrow$  making the algorithm

## Algorithm efficiency & Analysis of an algorithm

- A measure of the average execution time required for an algorithm to complete work on a set of data.
- Algorithm efficiency is characterized by its order.
- There are 2 main measures of the efficiency of an algorithm

1. Time complexity :- How much time it consumes to execute the program
2. Space complexity :- How much amount of memory require by an algorithm during course of execution.

### Asymptotic notation

SN  
\* Mathematical way to represent a time complexity

There are 3 notations

1. Big-O  $\rightarrow$  worst case
2. Theta- $\Theta$   $\rightarrow$  average case
3. Omega- $\Omega$   $\rightarrow$  best case

$$f(n) = O(g(n))$$

$$f(n) \leq c \cdot g(n)^2$$

Big O

$$\text{Given } c=3, g(n)=5$$

$$f(n) = 2n^2 + n$$

$$f(n) \leq c \cdot g(n)^2$$

$$2n^2 + n \leq 3 \cdot (5)^2$$

$$2(5)^2 + 5 \leq 3 \cdot 25$$

$$2(25) + 5 \leq 75$$

definition

time

$$f(n) \leq c \cdot g(n)$$

growth rate  $\rightarrow$  greater growth rate.

Omega- $\Omega$

lower bound.

$$f(n) = \Omega(g(n))$$

formula

(5)

$$f(n) = 2n^2 + n \quad n=2 \text{ or } 5 \text{ (any value can be taken)}$$

$$\frac{f(n)}{g(n)} \geq c \cdot g(n)^2$$

$$2n^2 + n \geq 2 \cdot g(n)^2$$

$$2(4) + 2 \geq 2 \cdot (2)^2$$

$$10 \geq 8$$

Theta- $\Theta$

$$f(n) \leq c_1 \cdot g(n)^2 \leq f(n) \leq c_2 \cdot g(n)^2$$

$$\text{Given: } f(n) = 2n^2 + n, n=5, c_1=2$$

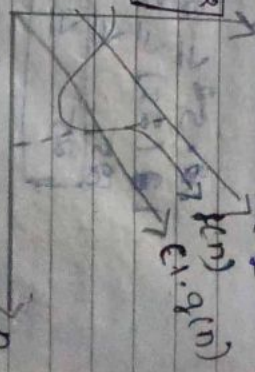
$$c_2=3$$

$$2 \cdot 5^2 \leq 2n^2 + n \leq 3 \cdot (5)^2$$

$$2 \cdot (5)^2 \leq 2(5^2) + 5 \leq 3 \cdot (5)^2$$

$$2 \cdot (25) \leq 2(25) + 5 \leq 3 \cdot (25)$$

$$50 \leq 55 \leq 75$$



Average case, except time, growth rate  $\rightarrow$  same growth

Big-O

Theta

Omega

If has a function It has a  $f^n$ It has a  $f^n$ 

formula

 $f(n) \leq g(n)$  $f(n) \leq g(n)$  $f(n) \leq g(n)$ 

Eqn :-

formula

 $f(n) = 0, g(n)$  $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$  $f(n) = 0, g(n)$  $f(n) \geq c \cdot g(n)$ 

growth rate slow

growth rate is equal

growth rate is fast

always greater than always equal always less than

(2)

Given:  $f(n) = 8n^3 + n$ ,  $n = 5$ ,  $c_1 = 1$ ,  $c_2 = 4$ formula:  $[c_1 \cdot g(n)^2 \leq f(n) \leq c_2 \cdot g(n)^2]$  $1 \cdot 5^2 \leq 8 \cdot 5^3 + 5 \leq 4 \cdot 5^2$  $1 \cdot (25) \leq 2(25) + 5 \leq 4(25)$  $25 \leq 50 + 5 \leq 100$  $25 \leq 55 \leq 100$ 

Definition

(1) Big-O :- Big-O notation has two functions

 $f(n)$  and  $g(n)$ 

we can say that Big-O notation

always less than. It means

growth rate is slow.

(2) Omega :- Omega notation has 2

functions  $f(n)$  &  $g(n)$ 

Omega is always greater than it means growth rate is fast.

(3) Theta :- Theta notation has 2

functions  $f(n)$  &  $g(n)$  is always equal (same) growth rate.

(2) algorithm &amp; characteristics

(2) what is asymptotic their types  $\{ \infty, \infty \}$ 

FUNDAMENTALS OF ALGORITHM.

(1) variable storing data value.

function - group of statement that together perform a task.

algorithm - standard input &amp; output

algorithm - standard input &amp; output

algorithm - standard input &amp; output

algorithm - standard input &amp; output

algorithm - standard input &amp; output

## FUNDAMENTALS OF ALGORITHM

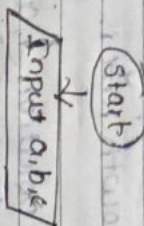
① Exchanging the value of 2 variables with algorithm

Algorithm  
Step 1: Start.  
Step 2: Input a, b, c.  
Step 3: c = a  
a = b  
b = c

Step 4: 0/p a, b

Step 5: Stop

Flowchart



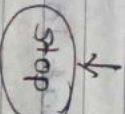
Program

```

c = a
a = b
b = c
  
```

```

output a, b
  
```



required

direct = 1/8, char = 1/5.

Standard input/output -> standard I/O

Integer -> %d (with no)

float -> %f (7.3/6.3)

char -> %c (name alphabet)

output = b=10, a=20.

a = 10  
b = 20

c = a  
a = b  
b = c

c = 10  
a = 20  
b = 10

c = 20  
a = 10  
b = 20

c = 10  
a = 20  
b = 10

c = 20  
a = 10  
b = 20

```

#include <stdio.h> // header file
#include <conio.h> // clrscr
void main() // starting of program
{
    int a, b, c; // declaration part
    printf("Enter a two numbers/n");
    scanf("%d %d", &a, &b);
    c = a;
    a = b;
    b = c;
    printf("a = %d, b = %d", a, b);
    getch();
}
  
```

```

scanf("%d %d", &a, &b);
// address specified
// scanf
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

```

getch();
// getch
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

output

Enter a two no;

a = 10, b = 20

value before swapping

value after swapping

b = 10, a = 20

OR

```

#include <stdio.h> // header file
#include <conio.h> // clrscr
void main() // main function
{
    int a, b, c; // local declaration
    printf("Enter a two no");
    scanf("%d %d", &a, &b);
    printf("a = %d, b = %d", a, b);
    c = a;
    a = b;
    b = c;
    printf("a = %d, b = %d", a, b);
    getch();
}
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

```

scanf("%d %d", &a, &b);
// scanf
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

```

printf("a = %d, b = %d", a, b);
// printf
  
```

② Exchanging the value of 2 variable without using third variable

Algorithm

a=10, b=5

Step 1: Start

Step 2: Input a, b, temp.

Step 3: temp = a = a+b

a = a-b

Step 4: output a, b.

Step 5: stop.

Flowchart

Start

Input a, b

a = a+b  
b = a-b

Output a, b

Stop

Ex: x=40, y=20

x = x+y

y = x-y

x = x-y

Program

#include <stdio.h>

#include <conio.h>

void main()

int a, b;

printf("Enter a two numbers/n");

scanf("%d %d", &a, &b);

a=a+b;

b=a-b;

a=a-b;

printf("Value after exchanging/n");

printf("a=%d, b=%d", a, b);

getch();

printf("The value after exchanging a=%d, b=%d",

③ Counting :- Sum and Product.

① Count the number of digits

Step 1: Start

Step 2: Read n

Step 3: count = 0

Step 4: while (n != 0)

Step 5: n = n/10

Step 6: Print count

Step 7: Stop

Start

Read n

count = 0

while n != 0

count++

n = n/10

Print count

Stop

n = 1834, c = 0

condition: while (n != 0)

n = n/10

count++

n = 1834, c = 0 (count++)

n = 183, c = 1 (count++)

n = 18, c = 2 (count++)

n = 1, c = 3 (count++)

n = 0, c = 3 (count++)

n = 0, c = 3 (count++)

Print count

Stop

While loop

Repeatedly carried out of no. of series of instructions till the condition is true.

② n = 3832, c = 0

condition: while (n != 0)

n = n/10

count++

n = 3832, c = 0

n = 383, c = 1

n = 38, c = 2

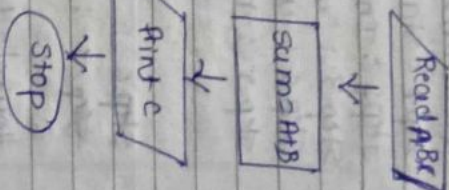
n = 3, c = 3

n = 0, c = 4

## ④ Sum of 2 numbers

### Algorithm

Step 1: Start  
 Step 2: Input A, B, C  
 Step 3: ~~Sum = A+B~~ Sum = A+B  
 Step 4: output C  
 Step 5: Stop.



### Program

```

#include <stdio.h>
#include <conio.h>
void main()
{

```

```

    int A, B, C, sum;

```

```

    printf("Enter any two number/n");

```

```

    scanf("%d %d", &A, &B);

```

```

    sum = A+B;

```

```

    printf("The sum of two number/n");

```

```

    getch();
    printf("a=%d, b=%d", A, B);
}

```

```

printf("Sum of 2d", sum);

```

O/P:  
 Enter any 2 NO:  
 10 20

Sum = 30.

## 1) Program for count the number of digits

```

#include <stdio.h>
#include <conio.h>
void main()
{

```

```

    int n, count = 0;

```

```

    printf("Enter any number/n");

```

```

    scanf("%d", &n);

```

```

    while(n != 0)
    {

```

```

        n = n/10;

```

```

        count++;
    }

```

```

    printf("Total no. of digits: %d", count);
    getch();
}

```

O/P:  
 Enter any number  
 1234  
 Total 4 digit

1234 j=0

1234/10 C=0

123/10 C=1

12/10 C=2

1/10 C=3

(9/10) = C=4

0

Total 4 digit

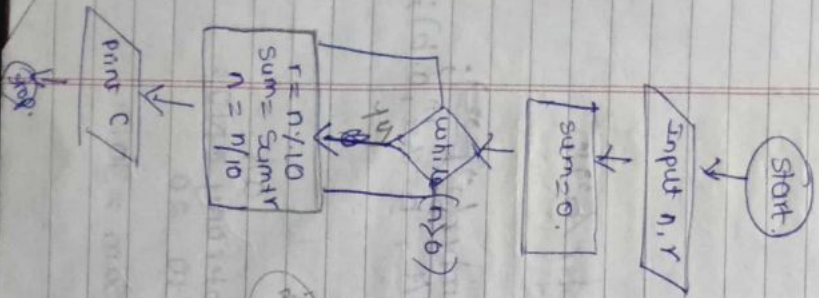
### Calculation

5) C program to find sum of digits of a number.

Algorithm:-

- Step 1: Start  
 Step 2: Input n, r.  
 Step 3: Sum = 0.  
 Step 4: while (n > 0)  
 Step 5:  $r = n \% 10$ ,  $Sum = Sum + r$   
 Step 6: Print sum  
 Step 7: Stop.

Flowchart



Calculation

$n = 123$ ,  $Sum = 0$   
 while (n > 0) then  
 $r = n \% 10$

$$\begin{array}{r} 10 \overline{) 123} \\ \underline{10} \phantom{0} \\ 23 \\ \underline{20} \\ 3 \end{array}$$
  
 $3 \rightarrow \text{remainder}$   
 $r = 3$

$Sum = Sum + r$   
 $Sum = 0 + 3 = 3$   
 $Sum = 3$

$n = 12$   
 $12 > 0$   

$$\begin{array}{r} 10 \overline{) 12} \\ \underline{10} \phantom{0} \\ 2 \end{array}$$
  
 $2 \rightarrow \text{remainder}$   
 $r = 2$

$Sum = Sum + r$   
 $Sum = 3 + 2 = 5$   
 $Sum = 5$

$n = 1$   
 $1 > 0$   

$$\begin{array}{r} 10 \overline{) 1} \\ \underline{0} \phantom{0} \\ 1 \end{array}$$
  
 $1 \rightarrow \text{remainder}$   
 $r = 1$

$Sum = Sum + r$   
 $Sum = 5 + 1 = 6$   
 $Sum = 6$   
 $r = 0$ ,  $n = 0$ ,  $0 > 0$

Program

```
#include <stdio.h>
// include <math.h>
void main()
```

```
int n, r, sum = 0;
printf("Enter any number\n");
scanf("%d", &n);
while (n > 0)
{
    r = n % 10;
    sum = sum + r;
    n = n / 10;
}
```

Print ("Sum of digits: %d", sum);  
 getch();

Calculation

$1234 > 0$   
 $r = n \% 10$   
 $Sum = Sum + r$   
 $Sum = 0 + r = 0 + 4 = 4$

$n = 123$   
 $123 > 0$   
 $r = 3$ ,  $4 + 3 = 7$   
 $n = 12$

$12 > 0$   
 $r = 2$ ,  $7 + 2 = 9$   
 $n = 1$

$1 > 0$   
 $r = 1$ ,  $9 + 1 = 10$   
 $n = 0$

Enter any number  
 1234

Sum digit: 10

$10 \overline{) 10}$   
 $\underline{10}$   
 $0$

$0 \rightarrow \text{remainder}$   
 $r = 0$

## Base conversion

### Number System:-

Number system is a way of representing or expressing the numbers.

or.

Represents a quantity through a set of numbers is called number system.

Ex:- I have \$100 in my wallet.

### Types of Number System:-

- ① Binary number system  $\rightarrow$  base 2
- ② Decimal number system  $\rightarrow$  base 10
- ③ Octal number system  $\rightarrow$  base 8
- ④ Hexadecimal Number System  $\rightarrow$  base 16

① Binary number system  
(0, 1) are binary number and base is 2

② Decimal number system  
base value is 10,  
numbers = (0-9)

③ Octal number system  
base is 8.  
numbers/digits = (0-7)

④ Hexadecimal Number system  
base = 16

digits = (0-9)  
letters = (A-F)

$$2^0 = 1$$

$$2^1 = 2$$

# ① BINARY NUMBER SYSTEM.

Binary number system is a number system  
base value is 2. (0,1)

Ex: 10101

10101

$$= 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$= 16 + 0 + 4 + 0 + 1$$

$$= 21$$

## ② DECIMAL NUMBER

base value is 10  
digits start from (0-9)

Ex: 2586

$$= 2 \times 10^3 + 5 \times 10^2 + 8 \times 10^1 + 6 \times 10^0$$

$$= 2000 + 500 + 80 + 6$$

$$= 2586$$

## ③ OCTAL NUMBER SYSTEM

base value is 8  
digits start from (0-7)

Ex: 2078

$$= 2 \times 8^3 + 0 \times 8^2 + 7 \times 8^1 + 8 \times 8^0$$

$$= 8192 + 0 + 56 + 8$$

$$= 8256$$

$$= 1024 + 0 + 56 + 8$$

$$= 1088$$

## ④ HEXADECIMAL

base value is 16.  
digits start from (0-9)  
letters A, B, C, D, E, F

10 11 12 13 14 15

Ex: 1AF<sub>16</sub>

$$= 1 \times 16^2 + A \times 16^1 + F \times 16^0$$

$$= 1 \times 256 + 10 \times 16 + 15 \times 1$$

$$= 256 + 160 + 15$$

$$= 431$$

## Base Conversion.

### ① Binary to decimal.

Ex: 101101<sub>2</sub> to ( )<sub>10</sub>

$$= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$= 64 + 0 + 16 + 8 + 0 + 1$$

$$= 90$$

101101<sub>2</sub> = 90<sub>10</sub>

### ② 1011<sub>2</sub> to ( )<sub>10</sub>

$$= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$= 8 + 0 + 2 + 1$$

$$= 11$$

# Base conversion Decimal to Binary

1  $90_{(10)}$  to binary

|     |      |      |
|-----|------|------|
| $2$ | $45$ | $-0$ |
| $2$ | $22$ | $-1$ |
| $2$ | $11$ | $-0$ |
| $2$ | $5$  | $-1$ |
| $2$ | $2$  | $-1$ |
| $1$ | $-0$ |      |

$90_{(10)} = 1011010_{(2)}$

2  $110_{(10)}$  to  $(2)$

|     |      |      |
|-----|------|------|
| $2$ | $55$ | $-1$ |
| $2$ | $27$ | $-1$ |
| $2$ | $13$ | $-0$ |
| $1$ | $-0$ |      |

$110_{(10)} = 1011101_{(2)}$

Decimal to Binary

Decimal to Octal

Decimal to Hexa

$117_{(10)}$  to  $(2)$

method - 01

|     |      |      |
|-----|------|------|
| $2$ | $58$ | $-1$ |
| $2$ | $29$ | $-0$ |
| $2$ | $14$ | $-1$ |
| $2$ | $7$  | $-0$ |
| $2$ | $3$  | $-1$ |
| $1$ | $-1$ |      |

$1110101_{(2)}$

method 2

|     |      |      |
|-----|------|------|
| $8$ | $14$ | $-5$ |
| $8$ | $6$  | $-2$ |
| $1$ | $-5$ |      |

$117_{(10)} = 165_{(8)}$

$117_{(10)} = 75_{(16)}$

method 2

|       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|
| $2^0$ | $2^1$ | $2^2$ | $2^3$ | $2^4$ | $2^5$ |
| $1$   | $2$   | $4$   | $8$   | $16$  | $32$  |
| $1$   | $1$   | $1$   | $1$   | $1$   | $1$   |

$= 64 + 32 + 16 + 4 + 1 = 117$

1) Decimal to binary 2) Binary to decimal 3) Binary to hexadecimal

1) 64 (10) to 2

64 (10) to 2

64 (10) to 16

$$\begin{array}{r} 64 \\ 32 \quad - \\ 32 \quad - \\ 16 \quad - \\ 16 \quad - \\ 8 \quad - \\ 8 \quad - \\ 4 \quad - \\ 4 \quad - \\ 2 \quad - \\ 2 \quad - \\ 1 \quad - \\ 1 \quad - \end{array}$$

$$\begin{array}{r} 64 \\ 32 \quad - \\ 32 \quad - \\ 16 \quad - \\ 16 \quad - \\ 8 \quad - \\ 8 \quad - \\ 4 \quad - \\ 4 \quad - \\ 2 \quad - \\ 2 \quad - \\ 1 \quad - \\ 1 \quad - \end{array}$$

40 (10)

1000000 (2)

2) Binary to decimal Binary to decimal

1) 10101 (2) to (10)

1) 10101 (2) to (10)

10101 (2) to (10)

10101 (2) to (10)

$$1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 16 + 0 + 4 + 0 + 1 = 21$$

$$1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 16 + 0 + 4 + 0 + 1 = 21$$

10101 (2) to (10)

10101 (2) to (10)

10101 (2) to (10)

10101 (2) to (10)

10101 (2) to (10)

10101 (2) to (10)

10101 (2) to (10)

10101 (2) to (10)

10101 (2) to (10)

10101 (2) to (10)

V.P.

10101

$$\begin{array}{r} 10101 \\ 10101 \\ 10101 \\ 10101 \\ 10101 \end{array}$$

#include <stdio.h>

#include <stdlib.h>

int main()

{

int a[10];

printf("Enter the num: ");

scanf("%d", &a[0]);

printf("Enter the 2nd num: ");

scanf("%d", &a[1]);

printf("Enter the 3rd num: ");

scanf("%d", &a[2]);

printf("Enter the 4th num: ");

scanf("%d", &a[3]);

printf("Enter the 5th num: ");

scanf("%d", &a[4]);

printf("Enter the 6th num: ");

scanf("%d", &a[5]);

printf("Enter the 7th num: ");

scanf("%d", &a[6]);

printf("Enter the 8th num: ");

scanf("%d", &a[7]);

printf("Enter the 9th num: ");

$$\begin{array}{r} 10101 \\ 10101 \\ 10101 \\ 10101 \\ 10101 \end{array}$$

543 to 9.16

1st step - 543

101 100 011  
4+1 4 2+1

16 8 4 2 1

2-100

5-2101 44)

$$8 \times 100 = 800$$

11-91011

2015

octal to hexadecimal.

$\frac{1}{6} \times 8 = \frac{4}{3}$

10

## Octal to binary

1

1141

---

---

$2^4$  2 1 2 2 1 2 4 2 1  
 00001100101

4-q10u p

Q19-7

---

---

1000

---

---

1

---

1000

9444

---

11

[illegible]

11

1

100

18

00

1041

1

5

1

1



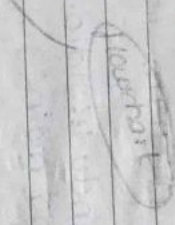
V.P

# Array Reverse

method 02

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int a[10], i, n;
    printf("Enter the limit:");
    scanf("%d", &n);
    printf("Enter %d numbers:", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("Reversed numbers are:");
    for (i = n-1; i >= 0; i--)
        printf("%d\n", a[i]);
    getch();
}
```

AN



method 01

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int a[10], i, n, j = 0;
    printf("Enter the limit:");
    scanf("%d", &n);
    printf("Enter %d numbers:", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("Reversed numbers are:");
    for (i = n-1; i >= 0; i--)
        printf("%d\n", a[i]);
    getch();
}
```

## Lab Program 15

### Array Techniques

#### Array Order Reversal.

##### Algorithm

Input: An integer array A and its size n  
Output: The reversed Array Rev

Step 1: Start.

Step 2: Read n

Step 3: Create an array A of size n

Step 4: for i = 1 to n  
Read A[i]

end for

Step 5: for i = 0 to n

Display A[i]

end for

Step 6: Create an array Rev of size n

Step 7: Read j = 1

Step 8: for i = n-1 to 0

Rev[j] = A[i]

j++

end for

Step 9: for i = 0 to n

Display Rev[i]

end for

Step 10: Stop.

# II FACTORING METHODS

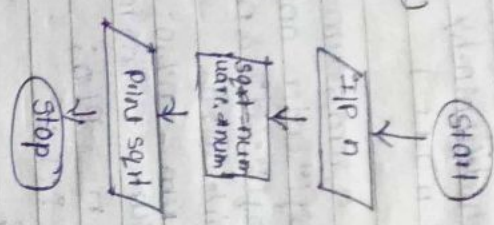
2. concept - Finding square root of a number.  
[Square of a number]

Algorithm

- Step 1: Start
- Step 2: Input number (num)
- Step 3:  $sq = num * num$
- Step 4: Print square.
- Step 5: Stop.

Calculation  
 $5 = 5 * 5 = 25$

Flowchart



Program to find square of a number

```

#include <stdio.h>
#include <math.h>
void main()
{
    int n, square;
    printf("Enter any number\n");
    scanf("%d", &n);
    sq = n * n;
    printf("The sq of given number is %d", sq);
    getch();
}
  
```

## Lab Program 9

Square root of a number.

ex:  $\sqrt{1} = 1$   
 $\sqrt{9} = 3$   
 $\sqrt{4} = 2$

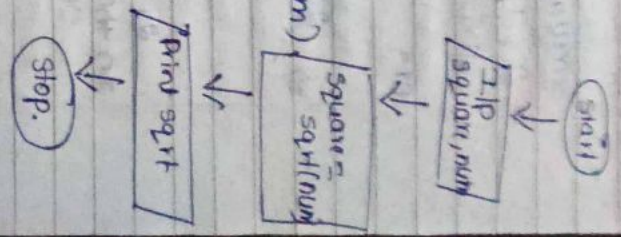
Algorithm:

- Step 1: Start
- Step 2: Input square, num
- Step 3:  $sq = \sqrt{num}$
- Step 4: Print square, sq
- Step 5: Stop.

Program

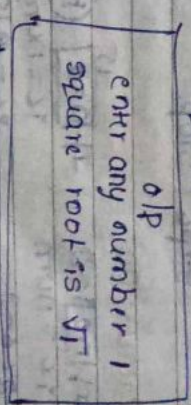
```

#include <stdio.h>
#include <math.h>
void main()
{
    int num, square;
    printf("Enter a number\n");
    scanf("%d", &num);
    square = sqrt(num);
    printf("Square root is %d", square);
    getch();
}
  
```



number multiplied by itself  
 $1 \times 1 = 1$   
 $3^2 = 3 \times 3 = 9 = \sqrt{9}$

without built-in function  
 $sq = n / 2$   
 $sq = (n + temp) / 2$



Concept

Q: Smallest divisor of an integer.

Smaller divisor of even numbers are divisible by 2, if n is even then 2 is the smallest divisor.

e.g.,  $n = 28$

$$\begin{array}{r} 14 \\ 2 \overline{) 28} \\ \underline{40} 8 \\ 28 \\ \underline{0} 0 \end{array}$$

$$\begin{array}{r} 7 \\ 4 \overline{) 28} \\ \underline{28} 0 \\ 0 \end{array}$$

$$\begin{array}{r} 4 \\ 7 \overline{) 28} \\ \underline{28} 0 \\ 0 \end{array}$$

$$\begin{array}{r} 2 \\ 14 \overline{) 28} \\ \underline{28} 0 \\ 0 \end{array}$$

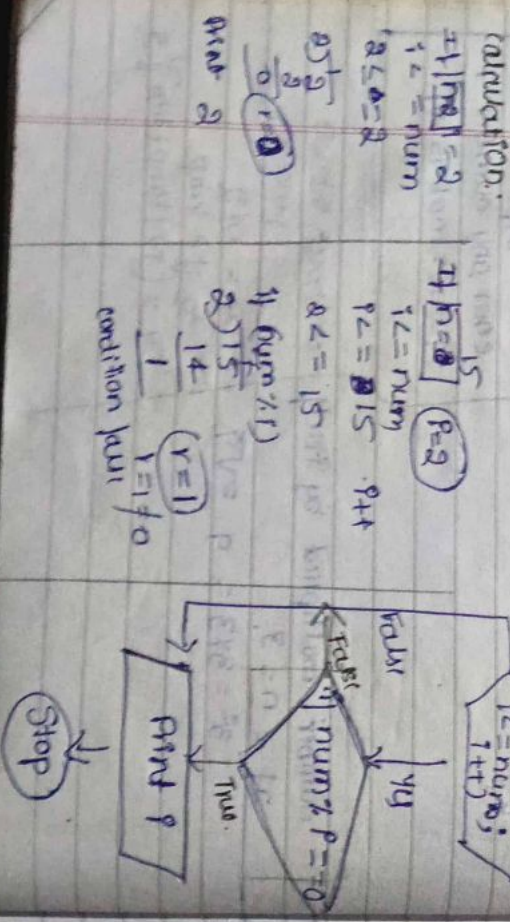
2, 4, 7, 14, 28 are the divisor

in this smallest divisor is 2

Algorithm

Step 1: Start  
 Step 2: Input num, i  
 Step 3: for (i = 2; i <= num; i++)  
 Step 4: if (num % i == 0)  
 Step 5: Print i  
 Step 6: Stop

Flowchart



if it is increment

$$\begin{array}{r} 15 \\ 3 \overline{) 15} \\ \underline{15} 0 \\ 0 \end{array}$$

$$\begin{array}{r} 5 \\ 15 \overline{) 15} \\ \underline{15} 0 \\ 0 \end{array}$$

$$\begin{array}{r} 3 \\ 15 \overline{) 15} \\ \underline{15} 0 \\ 0 \end{array}$$

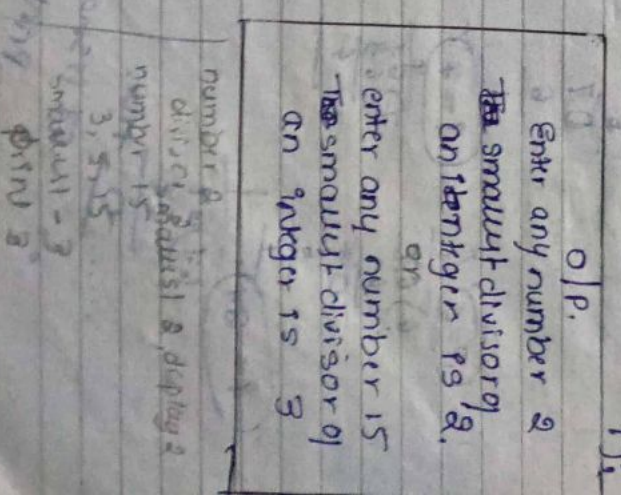
∴ 3 is the smallest divisor

Program

```

#include <stdio.h>
#include <conio.h>
void main()
{
  int num, p;
  printf("Enter any number\n");
  scanf("%d", &num);
  for (p = 2; p <= num; p++)
  {
    if (num % p == 0)
    {
      printf("Smallest divisor of an integer is %d", p);
      break;
    }
  }
  getch();
}

```



1)  $n = 67$

$p = 2$   
condition -  $p \leq \text{num}$   
 $8 \leq 67$

$8 \leq 67$

1) (num % 2)

$67 \div 2 = 33$   
 $2 \div 67$

$07$   
 $6 \div 7 = 1$

$r = 1 \neq 0$

2)  $p = p + 1$   
 $p = 3$   
condition -  $p \leq \text{num}$   
 $3 \leq 67$

$67 \div 3 = 22$   
 $3 \div 67$

$07$   
 $6 \div 7 = 1$

$p = 6$   
condition -  $p \leq \text{num}$   
 $6 \leq 67$

$67 \div 6 = 11$   
 $6 \div 67$

$p = 7$   
condition -  $p \leq \text{num}$   
 $7 \leq 67$

$67 \div 7 = 9$   
 $7 \div 67$

$07$   
 $6 \div 7 = 1$

$67$   
 $6 \div 7 = 1$

Greatest Common Divisor (GCD) of two numbers (input)

The greatest common divisor of two numbers is the greatest common factor number that divides them exactly. It is also called highest common factor.

$3 \div 15 = 5$   
 $2 \div 10 = 5$   
GCD = 5

Since both the numbers can be divide by 5

$5 \div 45 = 9$   
 $5 \div 25 = 5$   
GCD = 5

$3 \div 12 = 4$   
 $3 \div 26 = 8$   
GCD = 2,  $2 \times 3 = 6$

$2 \times 3 = 6$   
 $6 \div 12 = 2$   
 $6 \div 26 = 4$   
GCD = 6

Program - 1  
Write a program to calculate GCD of two numbers

Algorithm:

Step 1: Start

Step 2: Input  $n_1, n_2$

Step 3:  $n1 = n1 \wedge n2$  (if  $n1 = n2$ )

Step 4:  $n1 = n1 \wedge n2$  (if  $n1 > n2$ )

Step 5:  $n1 = n1 \wedge n2$  (if  $n1 < n2$ )

else

$n2 = n2 - n1$  (go to step 4.)

Step 6: Print  $n$

Step 7: Stop

# Prime Numbers

Prime numbers are natural numbers that are divisible by only 1 and the number itself. For prime numbers, 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 71, 73, 79, 83, 89, 97 (1-100 prime numbers)

## Lab program - 8

Write a C program to generate prime numbers

### Algorithm

- Step 1: Start
- Step 2: Input n
- Step 3: Let c = 0 (count = 0)
- Step 4: for (i = 2; i ≤ n; i++) (count = 0) c++

for (j = 1; j ≤ i; j++)  
if (i % j == 0)  
c++

Step 5: if (count == 2)

Step 6: print

Step 7: stop

### Calculation

n = 5 (i = 2; i ≤ 5; i++)  
count = 0  
j = 1 (j ≤ i; j++)  
if (i % j == 0) count++  
if (count == 2) print i  
if (count == 2) print i

i = 2, j = 1  
if (2 % 1 == 0) count++  
count = 1  
if (count == 2) print i  
if (count == 2) print i

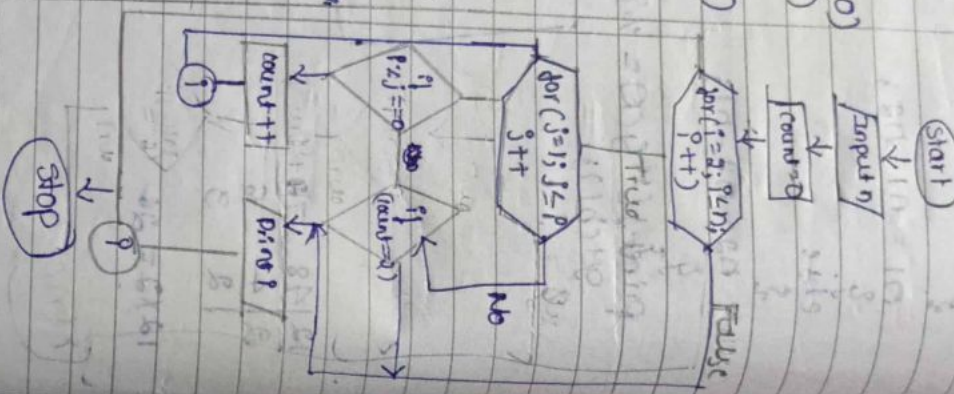
i = 3, j = 1  
if (3 % 1 == 0) count++  
count = 1  
if (count == 2) print i  
if (count == 2) print i

i = 4, j = 1  
if (4 % 1 == 0) count++  
count = 1  
if (count == 2) print i  
if (count == 2) print i

i = 5, j = 1  
if (5 % 1 == 0) count++  
count = 1  
if (count == 2) print i  
if (count == 2) print i

count = 2  
print i  
count = 2  
print i

### Flowchart



## Lab Program - 8

Program to generate prime numbers

```
#include <stdio.h>
#include <conio.h>
void main()
```

```
{
    int n, count = 0, i, j;
```

```
    printf("Enter any number\n");
    scanf("%d", &n);
```

```
    for (i = 2; i <= n; i++)
        printf("Prime numbers");
```

```
    for (j = 2; j <= i; j++)
        count = 0;
```

```
    for (j = 1; j <= i; j++)
```

```
    if (i % j == 0)
        count++;
```

```
    if (count == 2)
        printf("%d ", i);
```

```
    getch();
}
```



only when

i = 1

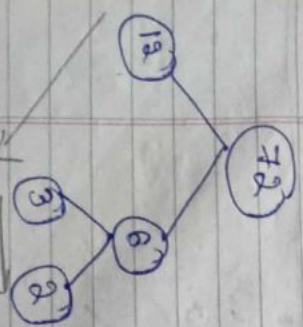
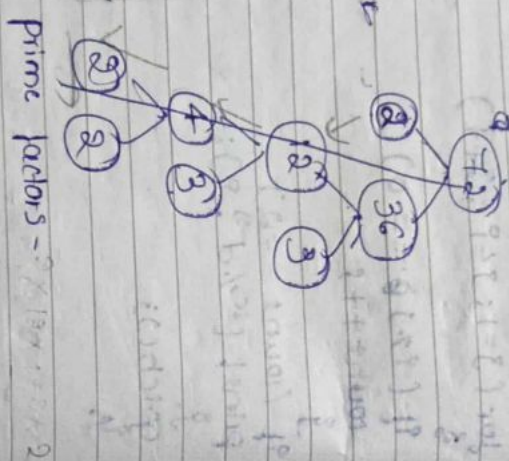
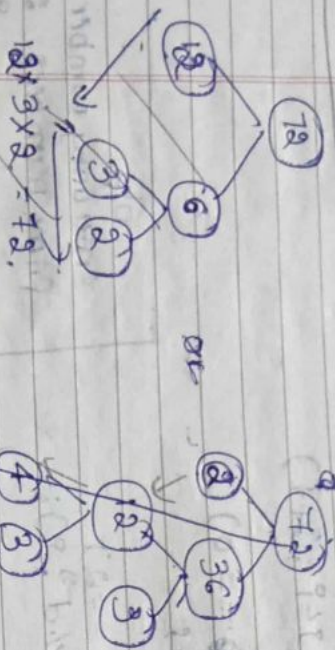
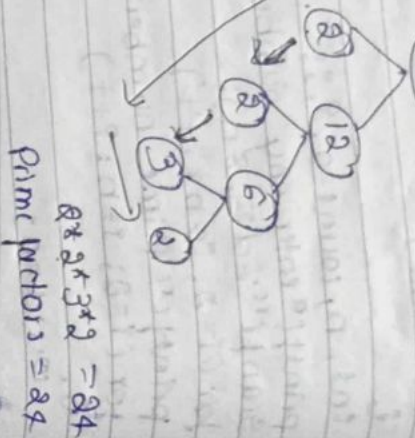
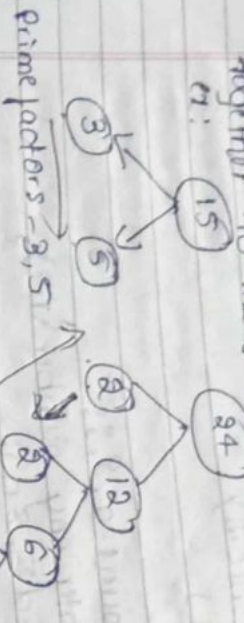
i = 3

loop ends or return

only when

# Prime factors

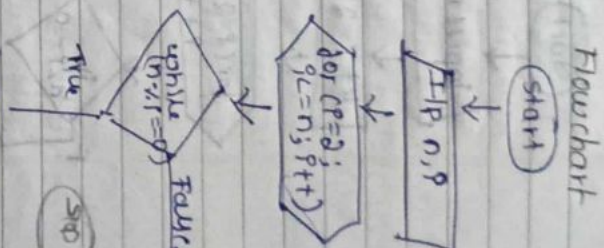
Prime factors are which prime numbers multiply together to make the original number.



12 = 2 x 2 x 3

Algorithm

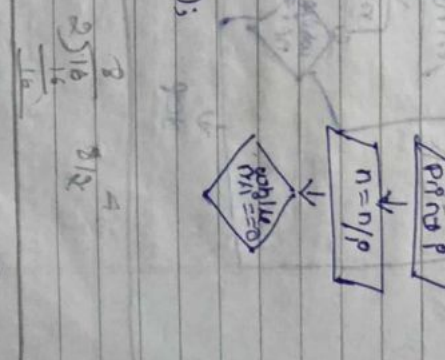
- Step 1: Start
- Step 2: Input n, p
- Step 3: for (i=2; i<=n; i++)
- Step 4: if (n%p == 0)
- Step 5: while (n/i % p == 0)
- Step 6: print p, n = n/p
- Step 7: stop



Program

```

#include <stdio.h>
#include <conio.h>
void main()
{
    int n, i;
    printf("Enter a number\n");
    scanf("%d", &n);
    for (i=2; i<=n; i++)
    {
        if (n%i == 0)
        {
            while (n%i == 0)
            {
                printf("%d", i);
                n = n/i;
            }
        }
    }
    getch();
}
  
```



while (n%i == 0)

enter a number 18

prime factor of a number 2x3x3

n = n/i

2x2x2x2

Generation of Pseudo random numbers  
Pseudo random numbers refers to an algorithm that uses mathematical formulae to produce a sequence of random numbers

Pseudo random numbers generate a sequence of numbers approximating the properties of random numbers

Characteristic / Properties of Pseudo random no.

→ ① Efficient

→ ② Deterministic

→ ③ LCM - Linear Congruential method

It is the most common algorithm for generating pseudo randomised numbers.

Formula:-

$$X_{i+1} = (aX_i + C) \text{ Mod } m.$$

$$RP = \frac{X_i}{m}$$

$X_i$  - is the seed element  
 $a$  - is the Multiplier

$C$  - is the Increment

$m$  - is the Modulus

$C \geq 0$  - Multiplicative LCM.  
 $C \neq 0 \rightarrow$  Mixed LCM.

ex:

Generating pseudo random numbers  
#include <stdio.h>  
#include <stdlib.h>  
void main()

```
{
    int i;
    i = rand();
    printf("%d\n", i);
    getch();
}
```

Using LCM generate a sequence of random numbers with  $X_0 = 27$ ,  $a = 17$ ,  $C = 43$   
 $\text{mod} = 100$

$$X_{i+1} = (aX_i + C) \text{ mod } 100$$

$$R_i = \frac{X_i}{m}$$

$$X_1 = (17 \cdot 27 + 43) \text{ mod } 100$$

$$= 459 + 43$$

$$= 502 \text{ mod } 100$$

$$X_1 = 0.502$$

$$X_2 = (17 \cdot 502 + 43) \text{ mod } 100$$

$$= 8534 + 43$$

$$= 8577 \text{ mod } 100$$

$$X_2 = 0.8577$$

$$X_3 = (17 \cdot 8577 + 43) \text{ mod } 100$$

$$= 145809 + 43$$

$$= 145852 \text{ mod } 100$$

$$X_3 = 0.95852$$

$$X_{i+1} = (aX_i + C) \text{ mod } m$$

$$R_i = \frac{X_i}{m}$$

$$RP = \frac{52}{100} = 0.52 //$$

$$x_4 = (17.52 + 43) \bmod 100$$

$$RP = \frac{87}{100}$$

$$= 884 + 43 \\ = 927 \bmod 100$$

$$x_4 = 27$$

$$RP = \frac{27}{100} = 0.27 //$$

$$x_5 = (17.27 + 43) \bmod 100$$

$$= 459 + 43 \\ = 502 \bmod 100$$

$$x_5 = \frac{2}{100} = 0.02$$

Raising a number to a large power.  
indicates how many copies of  
the base multiplied together =

$$E.g.: 3^5 = 3 * 3 * 3 * 3 * 3 = 243 //$$

The base 3 appears 5 times in  
multiplication because the exponent is  
5 and the 5th power of 3

$$4^4 = 4 * 4 * 4 * 4 = 256 //$$

$$3 \wedge 5$$

Binary power result

$$0 \quad x^1$$

$$0 \quad x^2 = x^2$$

$$1 \quad x^3 * x^2 = x^5$$

$$0 \quad x^4 * x^4 = x^8$$

$$1 \quad x^8 * x^8 = x^{16}$$

$$x^4 * x^{16} = x^{20}$$

$$x^4 + x^{16} = x^{20}$$

$$x_{16} = \frac{20}{1010}$$

$$0 * x^0 + 1 * x^1 + 0 * x^2 + 1 * x^3 \\ = 0 + 2 + 0 + 8$$

$$x_{10} = x^2 + x^8 //$$

- 1) What is algorithm?
- 2) What is the need for algorithm? (advantage)
- 3) What are the types of algorithm?
- 4) Merits & demerits of an algorithm
- 5) Advantages and disadvantages of algorithm
- 6) Define flow-chart, description of flowchart
- 7) What is the problem solving aspects?
- 8) What is the meaning of top down design with examples?
- 9) Why do we need top down design? explain advantage
- 10) Explain implementation of algorithm/program solving
- 11) Algorithm  
Flowchart  
Program  
Exchanging values of a variable
- 12) Explain counting integer in range
- 13) Flowchart  
Summation of 2 no  
factorial computation  
Fibonacci sequence 8th
- 14) Explain the concept of Base conversion
- 15) Explain concept of character to no conversion
- 16) Pseudocode, Asymptotic notation

Introduction to computer computer problem solving

1) Introduction - Fundamentals of Algorithm

2) What is algorithm?

3) Algorithm for sum of 2 no

4) Characteristics of algorithm

5) Application of algorithm

6) Computer science & software development

7) Healthcare → Diagnosis & treatment

8) Finance → Fraud detection, etc.

9) Engineering → control systems, etc.

10) AI, ML, etc. → Image recognition, etc.

11) Science & Research. Logistics, etc.

12) Everyday life → Navigation, Apps

13) Education → exam grading systems

14) Gaming & Entertainment

15) Basic steps for designing algorithm/Problem solving aspects / concept of problem solving

16) Problem solving in algorithm

17) Algorithm Design

18) Flowchart

19) Pseudocode

# 6 Top down design / stepwise refinement

Top down design also known as stepwise refinement, in which complex problems are broken down into smaller sub problems and each sub problem solved individually and integrating the solution to form complete algorithm

Key concept of Top down design  
Decomposition: where the large complex problems divided into smaller, simpler sub problems this process continues until each sub problem can be solved easily.

2 Modularisation: each sub problem can be treated as a separate module, thus modules can be re-used, developed, modified & revised

3 Hierarchy structure: it divides the whole problem into levels where top level represents overall problem and lower level represents sub problems

4 stepwise refinement: involves progressively refining each sub problem its continuous until each module can be fully developed.

Steps of Top down design

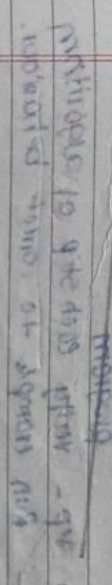
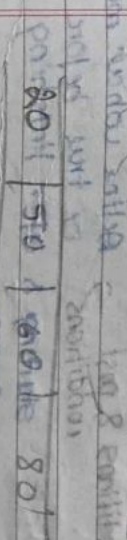
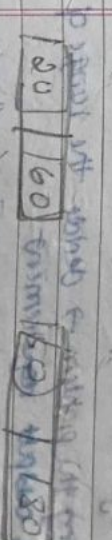
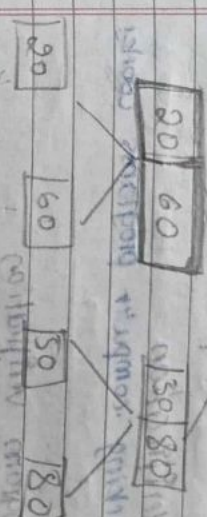
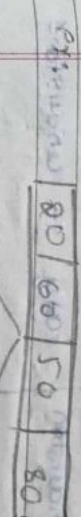
- 1 Identify main problem
- 2 Break down the problem into several sub problems
- 3 Refine sub problem
- 4 Design algorithm for sub problem → once all sub problems refined design algorithm for each sub problem
- 5 Implement solution

# 7 Testing & integration

Advantages of top down  
Clarity  
Modularity  
Maintainability  
Reliability

Disadvantages  
Rigidity  
Overlooking details  
Dependency

- \* easy to discover error
- \* well analysed & well designed
- \* leads to reliable program



- 1 Identify main problem
- 2 Decomposition
- 3 Modularisation
- 4 Hierarchy structure
- 5 Stepwise refinement
- 6 Design algorithm for sub problem
- 7 Implement solution
- 8 Testing & integration

AlgorithmDesigning

needs human knowledge  
can be written in any lang  
no need of H.W & S.W  
We analyse Algorithm

ProgramImplementing

~~needs~~ Programmer knows  
programming language  
need of H.W & S.W.  
we test Algorithm

Algorithm

simple english used  
easy to modify

complex to understand

Flowchart

symbols are used  
difficult to modify

easy to understand

Program Verification

Process of checking computer programs works correctly

Steps in Program Verification

- ① Understanding the problem → decide the range of inputs & output requirement
- ② Set pre conditions & post conditions → Define what must be true before starting & after finishing program
- ③ check each step - verify each step of algorithm
- ④ Test case - Run example to correct behaviour.